

手摸手实现规约组件模块

组件地址

```
<dependency>
  <groupId>org.opengoofy.index12306</groupId>
  <artifactId>index12306-convention-spring-boot-starter</artifactId>
  <version>${project.version}</version>
</dependency>
```

组件概述

1. 定义异常码抽象和公共异常码。
2. 定义抽象异常和客户端、服务端以及远程调用异常。
3. 封装分页对象请求参数和返回参数实体。
4. 封装公共响应对象。

组件功能

异常码抽象和公共异常码

为什么要做异常码规范和抽象异常码？如果没有这些规范，大家项目中抛出异常是怎么样的？

假设不同项目中抛出了相同的业务异常，可能是这种场景：

```
# A项目
{
  "code": "1"
}

# B项目
{
  "code": "000001"
}
```

```
# C项目
{
  "code": "500"
}

# xxx项目
{
  "code": "xxx"
}
```

异常码的不统一，带来的最直接后果就是返回结果异常码混乱，没有规律，也不方便排查问题。

为此，我参考了阿里开发规约中，关于异常码的定义部分，最终抽象了异常码接口以及公共异常码。

1) 异常码接口抽象。

```
package org.opengoofy.index12306.framework.starter.convention.errorcode;

/**
 * 平台错误码
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public interface IErrorCode {

    /**
     * 错误码
     */
    String code();

    /**
     * 错误信息
     */
    String message();
}
```

2) 封装常用的公共异常码。

```

package org.opengoofy.index12306.framework.starter.convention.errorcode;

/**
 * 基础错误码定义
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public enum BaseErrorCode implements IErrorCode {

    // ===== 一级宏观错误码 客户端错误 =====
    CLIENT_ERROR("A000001", "用户端错误"),

    // ===== 二级宏观错误码 用户注册错误 =====
    USER_REGISTER_ERROR("A000100", "用户注册错误"),
    USER_NAME_VERIFY_ERROR("A000110", "用户名校验失败"),
    USER_NAME_EXIST_ERROR("A000111", "用户名已存在"),
    USER_NAME_SENSITIVE_ERROR("A000112", "用户名包含敏感词"),
    USER_NAME_SPECIAL_CHARACTER_ERROR("A000113", "用户名包含特殊字符"),
    PASSWORD_VERIFY_ERROR("A000120", "密码校验失败"),
    PASSWORD_SHORT_ERROR("A000121", "密码长度不够"),
    PHONE_VERIFY_ERROR("A000151", "手机格式校验失败"),

    // ===== 二级宏观错误码 系统请求缺少幂等Token =====
    IDEMPOTENT_TOKEN_NULL_ERROR("A000200", "幂等Token为空"),
    IDEMPOTENT_TOKEN_DELETE_ERROR("A000201", "幂等Token已被使用或失效"),

    // ===== 一级宏观错误码 系统执行出错 =====
    SERVICE_ERROR("B000001", "系统执行出错"),
    // ===== 二级宏观错误码 系统执行超时 =====
    SERVICE_TIMEOUT_ERROR("B000100", "系统执行超时"),

    // ===== 一级宏观错误码 调用第三方服务出错 =====
    REMOTE_ERROR("C000001", "调用第三方服务出错");

    private final String code;

    private final String message;

    BaseErrorCode(String code, String message) {
        this.code = code;
        this.message = message;
    }

```

```
}

@Override
public String code() {
    return code;
}

@Override
public String message() {
    return message;
}
}
```

这些异常码为什么会这么定义？都是什么意思？参考异常码定义章节。

[只会控制层捕获异常？可以尝试全局拦截](#)

抽象异常体系

参考 #异常如何进行分类章节。

[只会控制层捕获异常？可以尝试全局拦截](#)

封装分页对象

MyBatisPlus 分页对象已经足够好了，为什么还要单独封装分页对象？

如果是我刚开始开这个项目，我可能会有这样的疑问。在已经比较完善的框架上封装新的功能，这不是多此一举么？

对于业务比较单一公司来说，不需要考虑这种情况，但是如果公司业务比较多就会出现不同的场景。我觉得下面两个原因是我要封装分页对象的主要出发点：

- 提供出去的 API 包，包里有个对象用了分页对象。但是依赖这个包的客户端没有使用 MyBatisPlus，难道要因为一个分页对象，把整个 MyBatisPlus 包都依赖进去么？
- 随着现在技术架构发展日新月异，谁也不能保证哪个框架可以长久留存。如果强依赖 MyBatisPlus，后续如果换了 ORM 框架，对于整体修改是个灾难。

基于这两点考虑，我抽象了分页入参和分页出参两个实体对象。

1) 分页请求参数对象。

```
package org.opengoofy.index12306.framework.starter.convention.page;

import lombok.Data;

/**
 * 分页请求对象
 *
 * <p> {@link PageRequest}、{@link PageResponse}
 * 可以理解为防腐层的一种实现，不论底层 ORM 框架，对外分页参数属性不变
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Data
public class PageRequest {

    /**
     * 当前页
     */
    private Long current = 1L;

    /**
     * 每页显示条数
     */
    private Long size = 10L;
}
```

2) 分页返回参数对象。

```
package org.opengoofy.index12306.framework.starter.convention.page;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.io.Serializable;
import java.util.Collections;
import java.util.List;
```

```

import java.util.function.Function;
import java.util.stream.Collectors;

/**
 * 分页返回对象
 *
 * <p> {@link PageRequest}、{@link PageResponse}
 * 可以理解为防腐层的一种实现，不论底层 ORM 框架，对外分页参数属性不变
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class PageResponse<T> implements Serializable {

    private static final long serialVersionUID = 1L;

    /**
     * 当前页
     */
    private Long current;

    /**
     * 每页显示条数
     */
    private Long size = 10L;

    /**
     * 总数
     */
    private Long total;

    /**
     * 查询数据列表
     */
    private List<T> records = Collections.emptyList();

    public PageResponse(long current, long size) {
        this(current, size, 0);
    }

```

```

    }

    public PageResponse(long current, long size, long total) {
        if (current > 1) {
            this.current = current;
        }
        this.size = size;
        this.total = total;
    }

    public PageResponse setRecords(List<T> records) {
        this.records = records;
        return this;
    }

    public <R> PageResponse<R> convert(Function<? super T, ? extends R> mapper) {
        List<R> collect =
this.getRecords().stream().map(mapper).collect(Collectors.toList());
        return ((PageResponse<R>) this).setRecords(collect);
    }
}

```

3) 提供 MyBatisPlus 分页和自定义分页对象之间的转换工具类，帮助开发提高开发效率。

```

package org.opengoofy.index12306.framework.starter.database.toolkit;

import com.baomidou.mybatisplus.core.metadata.IPage;
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import org.opengoofy.index12306.framework.starter.common.toolkit.BeanUtil;
import org.opengoofy.index12306.framework.starter.convention.page.PageRequest;
import org.opengoofy.index12306.framework.starter.convention.page.PageResponse;

import java.util.List;
import java.util.function.Function;
import java.util.stream.Collectors;

/**
 * 分页工具类
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */

```

```

public class PageUtil {

    /**
     * {@link PageRequest} to {@link Page}
     */
    public static Page convert(PageRequest pageRequest) {
        return convert(pageRequest.getCurrent(), pageRequest.getSize());
    }

    /**
     * {@link PageRequest} to {@link Page}
     */
    public static Page convert(long current, long size) {
        return new Page(current, size);
    }

    /**
     * {@link IPage} to {@link PageResponse}
     */
    public static PageResponse convert(IPage iPage) {
        return buildConventionPage(iPage);
    }

    /**
     * {@link IPage} to {@link PageResponse}
     */
    public static <TARGET, ORIGINAL> PageResponse<TARGET> convert(IPage<ORIGINAL> iPage,
Class<TARGET> targetClass) {
        iPage.convert(each -> BeanUtil.convert(each, targetClass));
        return buildConventionPage(iPage);
    }

    /**
     * {@link IPage} to {@link PageResponse}
     */
    public static <TARGET, ORIGINAL> PageResponse<TARGET> convert(IPage<ORIGINAL> iPage,
Function<? super ORIGINAL, ? extends TARGET> mapper) {
        List<TARGET> targetDataList = iPage.getRecords().stream()
            .map(mapper)
            .collect(Collectors.toList());
        return PageResponse.<TARGET>builder()

```



```
        .current(iPage.getCurrent())
        .size(iPage.getSize())
        .records(targetDataList)
        .total(iPage.getTotal())
        .build();
    }

    /**
     * {@link IPage} build to {@link PageResponse}
     */
    private static PageResponse buildConventionPage(IPage iPage) {
        return PageResponse.builder()
            .current(iPage.getCurrent())
            .size(iPage.getSize())
            .records(iPage.getRecords())
            .total(iPage.getTotal())
            .build();
    }
}
```

封装公共响应对象

参考该文档了解为什么封装公共响应对象。

[后端应用返回实体统一](#)

更新: 2023-09-17 23:45:51

原文: <<https://www.yuque.com/magestack/12306/mhds4epox1x2my7a>>